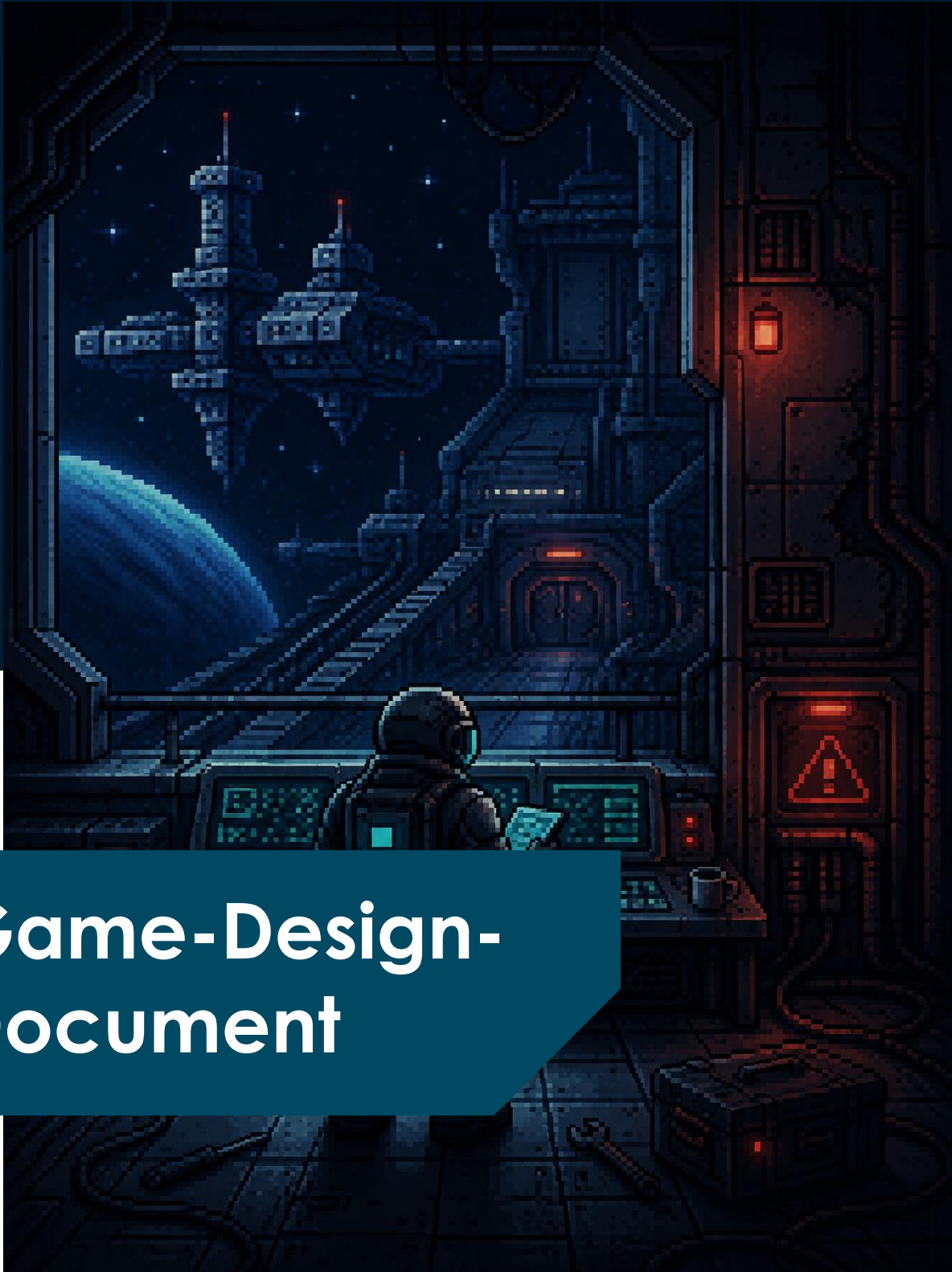




Game-Design-Document

Projektname: Deep Space Debugger

Ersteller: Franz Hielscher

INHALTSVERZEICHNIS

1.	PROJEKTÜBERSICHT	4
1.1	Projektbeschreibung	4
1.2	Mission Statement	4
1.3	Unique Selling Point	5
1.4	Zielgruppenanalyse	5
1.5	Ziele	7
1.6	Vision	8
2.	PROJEKTPLANUNG	9
2.1	Erläuterung zur Planung	9
2.2	Gantt Diagramm	10
3.	RISIKEN	11
3.1	Technische Herausforderungen	11
3.2	Zielgruppenakzeptanz	11
3.3	Zeitliche Risiken	12
4.	TECHNISCHE SPEZIFIKATION	13
4.1	Zielplattformen	13
4.2	Engine / Tools	13
4.3	Technologie: Objektorientierte Spielarchitektur	14
5.	AUDIO UND MUSIK	15
5.1	Soundeffekte	15
5.2	Musikstil	16
6.	GRAFIK UND ASSETS	17
6.1	Visueller Stil	17
6.2	Umgebungsdesign	17
6.3	Charakterdesign	18
6.4	Interaktive Elemente	18
7.	Interaktionsdesign	19
7.1	Controller-Input	19
7.2	Physische Mechaniken	20
8.	Spielerfahrung	24
8.1	Immersionselemente	24
8.2	Bewegungssystem	24
8.3	UI/UX Design	25

9.	Spielverlauf	26
9.1	Grundstruktur: Bereichsbasierter Fortschritt	26
9.2	Core-Game-Loop	27
9.3	Fortbewegung und Übergänge	27
9.4	Spielabschluss	28
10.	Story & Setting	29
10.1	Wichtige Orte	29
11.	MOODBOARDS UND IMPRESSIONEN	30
12	QUELLEN	32
12.1	Bildquellen	32

1. PROJEKTÜBERSICHT

1.1 Projektbeschreibung

Das Spiel **Deep Space Debugger** ist ein prototypisches **2D-Serious-Game** zur spielerischen Vermittlung grundlegender Konzepte des objektorientierten Paradigmas (OOP). Das Herzstück des Projekts ist eine interaktive Lernumgebung im **Science-Fiction-Setting einer beschädigten Raumstation**, in der Spielende verschiedene Systeme analysieren, reparieren und schrittweise wiederherstellen müssen.

Die Anwendung setzt auf eine klare **Verbindung von Gameplay, Rätseldesign und didaktischen Lernmechaniken**, um theoretische Inhalte des OOP verständlich und motivierend zu vermitteln. Das visuelle Fundament basiert auf einer **stilisierten 2D-Pixeloptik** und wird ergänzt durch:

- **Interaktive Debugging- und Rätselmechaniken.**
- **Gameplaybasierte Vermittlung von OOP-Konzepten** wie Klassen, Objekten, Vererbung und Polymorphie.
- **Spielerische Exploration und Freischaltung neuer Stationsbereiche.**
- **Atmosphärisches Audio- und UI-Design** zur Unterstützung der Immersion.

Der Fokus liegt auf der spielerischen Unterstützung von Lernprozessen durch interaktives Problemlösen und Anwendung objektorientierter Prinzipien. Ziel des Projekts ist die prototypische Entwicklung eines motivierenden Lernansatzes, der theoretische Programmierkonzepte durch direkte spielerische Erfahrung greifbar macht.

1.2 Mission Statement

Deep Space Debugger verfolgt das Ziel, grundlegende Konzepte der objektorientierten Programmierung durch **spielerische Interaktion** verständlicher und motivierender zu vermitteln. Durch exploratives Gameplay, interaktive Rätselmechaniken und die praktische Anwendung von OOP-Prinzipien soll Studierenden der Zugang zu abstrakten Konzepten wie Klassen, Vererbung und Polymorphie erleichtert werden. Das Spiel versteht sich dabei als ergänzender Lernansatz zu klassischen Lehrmethoden und verbindet theoretische Inhalte mit aktiver Problemlösung in einer interaktiven Spielumgebung.

1.3 Unique Selling Point

Der besondere Ansatz von **Deep Space Debugger** liegt in der direkten Verknüpfung von Gameplay und der Anwendung des objektorientierten Paradigmas. OOP-Konzepte werden nicht nur erklärt, sondern aktiv als Spielmechaniken erfahrbar gemacht und in eine interaktive Science-Fiction-Lernumgebung eingebettet.

1.4 Zielgruppenanalyse

Ausgangslage

Die Zielgruppe ergibt sich aus der Problemstellung des Projekts. Studierende technischer und informatischer Studiengänge haben häufig Schwierigkeiten, abstrakte Konzepte der OOP nachhaltig zu verstehen und praktisch anzuwenden.

Primäre Zielgruppe: Studierende technischer und informatischer Studiengänge

Die primäre Zielgruppe umfasst Studierende, die sich im Rahmen ihres Studiums mit den Konzepten des OOP beschäftigen. Dazu zählen insbesondere:

- Informatikstudierende
- Medieninformatikstudierende
- Studierende technischer Studiengänge
- Teilnehmende an Programmierkursen

Sekundäre Zielgruppe: Selbstlernende und programmierinteressierte Personen

Zusätzlich richtet sich das Projekt an:

- Hobbyentwickler:innen
- Selbstlernende
- Personen mit Interesse am Game-Based-Learning
- Programmieranfänger:innen außerhalb akademischer Kontexte

Merkmal	Beschreibung	Relevanz für das Design
Alter	ca. 18 bis 36 Jahre	Zielgruppe ist mit digitalen Medien und Computerspielen vertraut; modernes und intuitives UI-Design erforderlich

Bildungskontext	Studierende, Selbstlernende und programmierinteressierte Personen	Lerninhalte müssen sowohl akademisch relevant als auch allgemein verständlich vermittelt werden
Vorwissen	Grundkenntnisse in Programmierung, Unsicherheit bei OOP-Konzepten	Komplexe Konzepte müssen visuell, interaktiv und schrittweise vermittelt werden
Tech-Affinität	Mittel bis hoch; Erfahrung mit Games, Software und digitalen Lernmedien	Ermöglicht gameplaybasierte Lernmechaniken und experimentelles Problemlösen
Spielerfahrung	Gelegenheitsspieler bis erfahrene Spieler	Steuerung und Gameplay sollten leicht verständlich bleiben, aber dennoch motivierende Interaktionen bieten
Lernstil	Visuell, interaktiv und explorativ	Rätsel, Exploration und direkte Systeminteraktion fördern „Learning by Doing“
Motivation	Lernen und Unterhaltung kombiniert	Serious-Game-Elemente sollen Motivation und Lernbereitschaft hinsichtlich OOP steigern
Typische Lernprobleme	Schwierigkeiten beim Verständnis abstrakter OOP-Konzepte	Gameplay muss theoretische Inhalte praktisch erfahrbar machen
Nutzungskontext	Studium, Selbstlernen oder ergänzende Lernunterstützung	Flexible und kompakte Spielerfahrung notwendig
Aufmerksamkeits-spanne	Begrenzte verfügbare Lernzeit	Kurze, kompakte Spielerfahrung (ca. 15 bis 20 Minuten) mit klarer Progression
Plattformnutzung	PC / Laptop	Kompatibilität für eine Vielzahl an Geräten

1.5 Ziele

Die Ziele leiten sich direkt aus der Mission ab, abstrakte Konzepte der objektorientierten Programmierung durch spielerische Interaktion verständlicher und motivierender zu vermitteln.

Didaktische Ziele (Game-Based-Learning & OOP)

- **Verständnis abstrakter OOP-Konzepte:** Spieler:innen sollen grundlegende Prinzipien des objektorientierten Paradigmas wie Klassen, Objekte, Vererbung und Polymorphie durch interaktive Spielmechaniken besser verstehen und anwenden können.
- **Praxisnaher Wissenstransfer:** Theoretische Inhalte sollen nicht nur textbasiert vermittelt, sondern durch gameplayorientierte Problemlösung direkt erfahrbar gemacht werden.
- **Learning by Doing:** Durch Exploration, Rätselmekaniken und Systeminteraktionen soll aktives und experimentelles Lernen gefördert werden.
- **Motivation und Lernmanagement:** Die Kombination aus Science-Fiction-Setting, Pixelart-Ästhetik und spielerischen Herausforderungen soll die Motivation steigern und den Zugang zu komplexen Lerninhalten erleichtern.
- **Niedrige Einstiegshürde:** Das Spiel soll OOP-Konzepte verständlich vermitteln, ohne umfangreiche Programmierkenntnisse oder klassische Codeeingabe vorauszusetzen.

Spielerische Ziele (Gameplay & Experience)

- **Interaktive Problemlösung:** Spieler:innen analysieren fehlerhafte Systeme, erkennen Zusammenhänge und reparieren logische Fehler innerhalb der Raumstation.
- **Exploration und Progression:** Neue Bereiche der Raumstation werden schrittweise durch gelöste Herausforderungen freigeschaltet.
- **Verknüpfung von Gameplay und Lerninhalten:** OOP-Konzepte sollen direkt in die Kernmechaniken des Spiels integriert werden.

Technologische Ziele (Entwicklung & Umsetzung)

- **Prototypische Umsetzung eines 2D-Serious-Games:** Entwicklung eines spielbaren Minimum Viable Products (MVP) mit zentralen Gameplay- und Lernmechaniken.
- **Modulare Systemarchitektur:** Aufbau eines objektorientierten Spielsystems, dass die vermittelten OOP-Konzepte auch technisch innerhalb der Softwarearchitektur widerspiegeln.

- **Technische Umsetzung in der Godot Engine:** Einsatz der Engine zur Entwicklung eines plattformunabhängigen 2D-Spiels mit Fokus auf Performance, Modularität und Erweiterbarkeit
- **Anwendung des MDA-Frameworks:** Strukturierte Konzeption der Mechanics, Dynamics und Aesthetics zur Verbindung von Gameplay, Lernerfahrung und Nutzerinteraktion.

1.6 Vision

Deep Space Debugger verfolgt die Vision, abstrakte Konzepte der objektorientierten Programmierung durch **interaktive und spielerische Lernansätze** zugänglicher zu machen. Das Projekt versteht sich als **prototypischer Beitrag** zur **Verbindung von Game-Based-Learning und Informatiklehre** und untersucht, wie spielerische Interaktion das Verständnis komplexer Programmierkonzepte unterstützen kann.

Langfristig soll gezeigt werden, wie Serious Games klassische Lehrmethoden sinnvoll ergänzen und Lernprozesse durch Motivation, Exploration und aktive Problemlösung fördern können.

2. PROJEKTPLANUNG

2.1 Erläuterung zur Planung

Die Projektplanung erfolgt strukturiert anhand definierter Arbeitspakete und Meilensteine. Dabei werden sowohl die wissenschaftliche Ausarbeitung der Bachelorarbeit als auch die prototypische Entwicklung des Serious Games **Deep Space Debugger** berücksichtigt.

Der Fokus liegt auf der schrittweisen Konzeption, technischen Umsetzung und Evaluation eines spielbaren Prototyps, der zentrale Konzepte der objektorientierten Programmierung durch gameplaybasierte Mechaniken vermittelt.

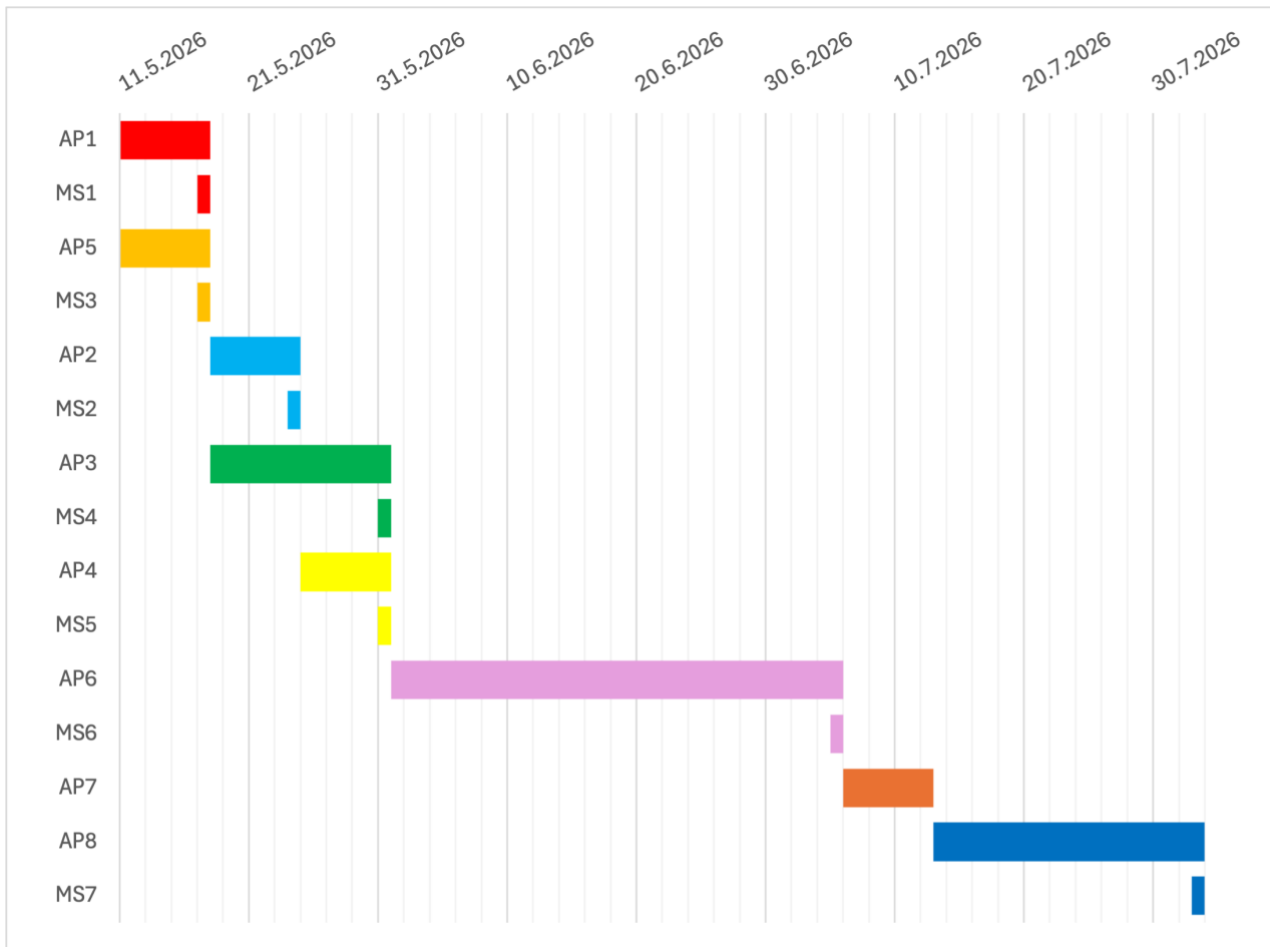
Meilensteine

- MS1 - Thema & Forschungs-/Spielkonzept definiert
- MS2 - Experteninterview durchgeführt & ausgewertet
- MS3 - Projektsetup (GitHub & Godot Engine) abgeschlossen
- MS4 - Literaturbasis erstellt & strukturiert
- MS5 - Game-Design-Dokument fertiggestellt
- MS6 - Spiel als spielbarer Prototyp (MVP) fertiggestellt
- MS7 - Abgabeverision der Bachelorarbeit fertiggestellt

Arbeitspakete

- AP1 - Planung & Organisation
- AP2 - Experteninterview
- AP3 - Literaturrecherche & Theorie
- AP4 - Game Design & Konzeption
- AP5 - Projektsetup (GitHub & Godot Engine)
- AP6 - Prototyp-Entwicklung (Mechaniken, grundlegende Spiellogik & Audio)
- AP7 - Testing & Optimierung
- AP8 - Finalisierung Bachelorarbeit

2.2 Gantt Diagramm



3. RISIKEN

Bei der prototypischen Entwicklung des Serious Games Deep Space Debugger ergeben sich verschiedene Risiken, die sowohl die **technische Umsetzung** als auch die **didaktische Gestaltung** sowie den **zeitlichen Projektverlauf** betreffen. Damit der Projekterfolg sichergestellt werden kann, ist es wichtig, dass **potenzielle Herausforderungen frühzeitig erkannt** und bei der **Planung berücksichtigt** werden. Die wesentlichen Risikobereiche umfassen technische Herausforderungen, Zielgruppenakzeptanz und zeitliche Risiken.

3.1 Technische Herausforderungen

Gameplay- und Systementwicklung:

- Fehleranfälligkeit bei komplexen Interaktionen zwischen verschiedenen Spielsystemen
- Bugs innerhalb der Rätsel-, Trigger- und Debugging-Mechaniken
- Fehlende Quellassets
- Schwierigkeiten bei der Umsetzung modularer OOP-basierter Spielsysteme

Technische Umsetzung in der Godot Engine:

- Probleme bei der Integration verschiedener Spielmechaniken
- Unerwartete Engine- oder Scriptfehler
- Risiko von Performanceproblemen bei zu vielen interaktiven Objekten

UI- und Debugging-System:

- Erhöhter Entwicklungsaufwand für ein verständliches UI- und Debugging-Interface
- Risiko einer zu umfangreichen und unübersichtlichen UI
- Komplizierte Balance zwischen Spielbarkeit und didaktischer Funktion

3.2 Zielgruppenakzeptanz

Vermittlung der Lerninhalte:

- Gefahr, dass die OOP-Konzepte weiterhin als zu abstrakt wahrgenommen werden
- Risiko, dass die Lerninhalte zu stark vereinfacht werden
- Unterschiedliche Vorkenntnisse innerhalb der Zielgruppen
- Kernmechaniken können die vorhandenen OOP-Konzepte nicht klar genug vermitteln

Motivation & Gameplay:

- Gefahr von Abbrüchen oder Nichtnutzung aufgrund fehlender Motivation
- Risiko, dass Spielmechaniken als zu repetitiv wahrgenommen werden
- Überforderung durch zu komplexe kombinierte Rätsel- und Lernmechaniken

Zugänglichkeit & Verfügbarkeit:

- Unterschiedliche Spielerfahrungen und technische Kenntnisse
- Risiko, dass die Steuerung oder Benutzeroberfläche zu komplex sind
- Begrenzte Zeitbereitschaft für Lernspiele innerhalb des Studienalltags

3.3 Zeitliche Risiken**Konzeption & Planung:**

- Hoher Aufwand bei Literaturrecherche und theoretischer Ausarbeitung
- Verzögerungen durch Anpassungen des Game-Designs
- Möglicher negativer Einfluss / zusätzliche Abstimmungen durch das geplante Experteninterview

Projektumfang:

- Risiko eines zu großen Scopes für die verfügbare Bearbeitungszeit
- Gefahr, dass geplante Features gekürzt / reduziert werden müssen
- Begrenzte Entwicklungszeit im Rahmen der Bachelorarbeit

Entwicklung, Optimierung & Testing:

- Zeitintensive Implementierung von Gameplaymechaniken und UI-Systemen
- Unerwarteter Aufwand bei der Fehlersuche und der Optimierung
- Umfangreiche Testphasen notwendig, um Fehler zu vermeiden

Externe Abhängigkeiten:

- Technische Probleme mit Entwicklungssoftware oder Hardware
- Zeitliche Überschneidung mit anderen Verpflichtungen
- Mögliche Verzögerungen durch Versions- oder Pluginprobleme innerhalb der Engine
- Mögliche Verzögerungen durch das Prüfungsamt

4. TECHNISCHE SPEZIFIKATION

4.1 Zielplattformen

Die Serious Game Deep Space Debugger wird als **plattformübergreifende 2D-Anwendung** mithilfe der **Godot Engine** entwickelt. Ziel ist eine möglichst flexible Nutzbarkeit auf verschiedenen Endgeräten, um den Zugang für unterschiedliche Zielgruppen zu gewährleisten.

Primäre Zielplattformen:

- Desktop-PCs und Laptops (Windows / MacOS / Linux)
- Steuerung über Tastatur und Maus

Kompatibilität & Erweiterbarkeit:

- Plattformunabhängige Entwicklung innerhalb der **Godot Engine**
- Fokus auf **modulare** und **skalierbare Projektstruktur**
- Möglichkeit zukünftiger Erweiterungen und zusätzlicher Plattformanpassungen im Rahmen weiterer Entwicklungsphasen

4.2 Engine / Tools

Game-Engine:

- **Godot Engine** (Nutzung der aktuellen Version, Stand: 11.05.2026) als zentrale Entwicklungsumgebung
- Programmierung primär mit **GScript**
- Entwicklung als **2D-Projekt** mit plattformübergreifender Unterstützung

Gameplay- & Systementwicklung:

- Nutzung des integrierten Node- und Szenensystems der Engine
- Modularer Aufbau der Spielsystem orientiert an OOP
- Entwicklung interaktiver Rätsel-, Trigger- und Debugging-Mechaniken

2D- und Asset-Erstellung / -Bearbeitung

- Erstellung und Bearbeitung von Pixelart-Grafiken
- Nutzung einfacher **2D-Asset-Pipelines** für Charaktere, Umgebungen und UI-Elemente

- Fokus auf stilisierte **Science-Fiction-Pixeloptik**

Audio-Tools:

- **Reaper / Audacity** zur Bearbeitung, Normalisierung und zum Export der Soundeffekte und Hintergrundmusik
- **Nutzung lizenzfreier Soundbibliotheken** (bspw. Freesound) als kostenfreie Quellen für Soundeffekte

Versionskontrolle:

- **GitHub** als primäres System für Versionsverwaltung und Projektorganisation

4.3 Technologie: Objektorientierte Spielarchitektur

Definition und Funktionsweise

Die technische Struktur von Deep Space Debugger basiert auf einer **modularen objektorientierten Spielarchitektur**. Zentrale Spielsysteme und Interaktionsobjekte werden dabei als eigenständige, wiederverwendbare Komponenten umgesetzt.

Das System orientiert sich an grundlegenden Prinzipien der objektorientierten Programmierung:

- **Klassen und Objekte**
- **Vererbung**
- **Polymorphie**
- **Objektbeziehungen und Komposition**

Interaktive Elemente wie Türen, Terminals oder Sicherheitssysteme besitzen **eigene Zustände und Verhaltensweisen** und reagieren **dynamisch** auf Spielerinteraktionen.

Nutzen für das Projekt

- Direkter Praxisbezug → Struktur spiegelt vermittelte OOP-Konzepte wider.
- Modulare Entwicklung & Wiederverwendbarkeit → flexible Erweiterung / Anpassung.
- Verständlichkeit → OOP-Konzepte werden spielerisch erfahrbar gemacht.

5. AUDIO UND MUSIK

5.1 Soundeffekte

Soundeffekte sollen das Gameplay unterstützen, Feedback vermitteln und die Atmosphäre des Raumstation-Settings verstärken. Dabei bleiben die Sounds funktional und zurückhaltend, um die Konzentration auf die Lern- und Rätselmechaniken nicht zu beeinträchtigen.

Typen von Soundeffekten:

- **Interaktionssounds:**
 - Aktivierung von Terminals
 - Öffnen und Schließen von Türen
 - Hochfahren von Systemen
 - Bestätigung von UI- und Debugging-Aktionen
- **System- und Umgebungsgeräusche:**
 - Elektrisches Summen von Maschinen und Energieversorgung
 - Warnsignale bei Fehlern und beschädigten Systemen
 - Fußstampfgeräusche bei Bewegung
 - Atmosphärische Raumstationsgeräusche und Hintergrundsounds
- **Gameplay-Feedback:**
 - Erfolgreiche / fehlgeschlagene Reparatur oder Aktivierung / Deaktivierung von Systemen
 - Fehlermeldungen bei falschen Interaktionen
 - Feedback bei Lösung von Rätseln und Fortschritt im Spielverlauf
- **Objektinteraktionen:**
 - Aktivieren, Verbinden oder Trennen von Modulen
 - Geräusche für Schalter, Terminals, Türen oder weiterer Sicherheitsmechanismen
 - Greifen, Anheben oder Nutzung von Objekten

5.2 Musikstil

Die Musik dient primär der atmosphärischen Unterstützung des Science-Fiction-Settings und soll die Spielenden während der Exploration und Problemlösung begleiten, ohne dabei vom Gameplay abzulenken.

Stil:

- Atmosphärische, ruhige Hintergrundmusik
- Elektronische und futuristische Klänge
- Minimalistische Ambient-Elemente
- Keine dominierenden Beats oder drastische Lautstärkewechsel
- Sanfte Überblendungen zwischen verschiedenen Spielsituationen und Bereichen

Zielsetzung:

- Unterstützung der Immersion
- Verstärkung / Förderung der Science-Fiction-Space-Atmosphäre
- Förderung einer konzentrierten und ruhigen Spielerfahrung für ideale Wissensaufnahme

6. GRAFIK UND ASSETS

6.1 Visueller Stil

Der visuelle Stil des Serious Games Deep Space Debugger orientiert sich an einer stilisierten 2D-Pixeloptik im Science-Fiction-Setting einer Raumstation. Ziel ist eine klare, atmosphärische und gut lesbare Darstellung der Spielwelt, die sowohl der Orientierung als auch die Vermittlung von Lerninhalten zum objektorientierten Paradigma unterstützt.

Zentrale Stilmerkmale:

- Stilisiertes 2D-Pixelart-Design
- Futuristische Science-Fiction-Umgebung mit Raumstationästhetik
- Klare Formen, gut erkennbare interaktive Elemente
- Einheitliche Material- und Farbgestaltung
- Fokus auf Lesbarkeit und Gameplay statt fotorealistischer Texturen
- Ruhige und übersichtliche visuelle Gestaltung zur Unterstützung der Konzentration

6.2 Umgebungsdesign

Das Umgebungsdesign basiert auf verschiedenen Bereichen einer beschädigten Raumstation. Die einzelnen Räume repräsentieren unterschiedliche Systeme und Gameplay-Schwerpunkte innerhalb des Spiels.

Schwerpunkte:

- Unterschiedliche Stationsbereiche mit eigenem visuellem und atmosphärischem Charakter
- Wiedererkennbare Science-Fiction-Elemente wie Terminals, Türen und Energiesysteme
- Klare Raumstrukturen zur Förderung der Orientierung
- Verwendung von leichtgewichtigen Assets zur Performanceoptimierung
- Atmosphärische Beleuchtung und technische Umgebungsdetails
- Fokus auf funktionale und gameplayrelevante Objekte in den verschiedenen Räumen

Mögliche Beispielbereiche:

- Schlafkapselraum
- Energieversorgung
- Kontrollraum für die Sicherheitssysteme
- Kommunikationszentrum
- Biosphären Farm

6.3 Charakterdesign

Der spielbare Charakter übernimmt die Rolle eines Systemtechnikers innerhalb der Raumstation. Das Charakterdesign bleibt bewusst reduziert und funktional, da der Fokus primär auf Gameplay, Exploration und Lernmechaniken liegt.

Zentrale Stilmerkmale:

- 2D-Pixelart-Figur
- Futuristische Arbeits- bzw. Raumstationsbekleidung (bspw. Raumanzug)
- Simple Animationen für Bewegung und Interaktionen
- Fokussierung auf Lesbarkeit und klare visuelle Darstellung

6.4 Interaktive Elemente

Interaktive Elemente sollen klar erkennbar, intuitiv verständlich und visuell hervorgehoben sein, um die Spielerführung und den Lernprozess zu unterstützen.

Mögliche Beispiele:

- Interaktive Terminals und Debugging-Stationen
- Türen, Sicherheitssysteme und Schalter
- Markierte Objekte für Rätsel- und Systeminteraktionen
- Minimalistische UI-Elemente und Informationsfenster
- Visuelle Hinweise / Veränderungen für aktivierte oder fehlerhafte Systeme
- Feedbackeffekte bei Nutzung von Interaktionen

7. INTERAKTIONSDSIGN

Das Interaktionsdesign verfolgt das Ziel, das objektorientierte Paradigma durch **intuitive Interaktionen** und **spielerische Problemlösung** zu vermitteln. Die Spieler*innen bewegen sich aus einer **Top-Down-Perspektive** durch eine Raumstation und **interagieren mit technischen Systemen, Robotern und Lernobjekten**. Der Fokus liegt auf **Exploration**, **Analyse technischer Probleme** und deren **Lösung durch OOP-inspirierte Mechaniken**.

7.1 Steuerung & Eingaben

Die Steuerung erfolgt vollständig über Tastatur und optional Maus.

Bewegung: W / A / S / D → Bewegung in alle Richtungen

Interaktion:

- Leertaste als Standardinteraktionstaste
 - Interaktion mit Objekten
 - Aktivierung von Terminals
 - Aufnahme & Ablegen von Objekten
 - Aktivierung von Schaltern / Buttons

Kontextbezogene Interaktionen:

- Interagierbare Objekte werden visuell hervorgehoben
- In der Nähe eines Objekts erscheint ein Hinweis (z.B. „Leertaste drücken“)
- Wichtige Objekte können durch Pulsieren oder andere Animationen hervorgehoben werden

Dialog- und Informationssystem:

- Dialog- / Textfelder werden automatisch oder durch Interaktion ausgelöst
- Informationen werden schrittweise über Textboxen vermittelt
- OOP-Konzepte werden direkt mit der aktuellen Spielsituation verknüpft

Feedback:

- Akustisches Feedback bei:
 - Interaktionen
 - Schalterbetätigung

- Türöffnungen
- Objektaufnahme und Ablage
- Visuelles Feedback durch:
 - Animationen
 - Hervorhebungen
 - Zustandsänderungen von Objekten

7.2 Spielmechaniken

Die Spielmechaniken orientieren sich am Core Gameplay Loop und unterstützen die Vermittlung des objektorientierten Paradigmas.

Exploration

- Raumstation besteht aus mehreren Bereichen
- Neue Räume werden schrittweise freigeschaltet
- Der Spieler entdeckt:
 - Beschädigte Systeme
 - Verschlussene Bereiche
 - Terminals
 - Gegner / Roboter
 - Lernobjekte

Interaktive Objekte

- Objekte können aufgenommen und transportiert werden
- Bestimmte Gegenstände werden zur Lösung von Problemen benötigt
- Beispiele:
 - Energiekern transportieren
 - Sicherheitsmodul einsetzen
 - Defekte Komponente austauschen

Schalter- und Türsysteme

- Schalter / Buttons aktivieren Systeme innerhalb der Station
- Türen öffnen neue Bereiche
- Einige Türen erfordern spezielle Bedingungen oder Gegenstände

Trigger-Systeme

- Triggerzonen lösen aus:
 - Dialoge
 - Hinweise
 - Lerninhalte
 - Ereignisse
 - Gegnerbegegnungen

Rätsel- und Problemlösung

- Technische Probleme bilden den Kern des Gameplays
- Spieler*in analysiert Situationen und nutzt OOP-Konzepte zur Lösung
- Beispiele:
 - Energienetz wiederherstellen
 - Sicherheitssystem reparieren
 - Roboter neu konfigurieren
 - Fehlfunktionen beheben

7.3 Lern- & OOP-Mechaniken

Die Vermittlung des objektorientierten Paradigmas erfolgt integriert in die Spielwelt und die Kernmechaniken von Deep Space Debugger. Die Spieler*innen lernen die Konzepte **nicht durch klassische Lehrtexte oder Quizfragen**, sondern durch das **Lösen technischer Probleme** innerhalb einer beschädigten Raumstation.

Der Lernprozess orientiert sich am Core Gameplay Loop:

- Erkunden der Raumstation
- Analyse eines technischen Problems
- Anwendung einer OOP-Mechanik
- Reparatur eines Systems oder Besiegen eines Gegners
- Erhalt einer Belohnung
- Freischaltung neuer Bereiche

Die einzelnen Konzepte der objektorientierten Programmierung werden dabei schrittweise eingeführt und mehrfach angewendet.

Objekte und Klassen

- Innerhalb der Raumstation begegnen die Spieler*innen verschiedenen technischen Geräten, Robotern und Systemen
- Mehrere Objekte können demselben Grundtyp zugeordnet werden
- Spielwelt verdeutlicht den Zusammenhang zwischen Klasse und Objekte
- Beispiele:
 - Mehrere Wartungsroboter basieren auf derselben Roboterklasse
 - Verschiedene Türen besitzen gemeinsame Grundfunktionen, unterscheiden sich jedoch in ihren Eigenschaften

Attribute

- Eigenschaften von Geräten und Objekten durch sichtbare Werte repräsentiert
- Spieler*innen müssen Eigenschaften analysieren und für die Problemlösung nutzen
- Beispiele:
 - Energiekerne mit unterschiedlicher Ladung
 - Sicherheitsmodule mit verschiedenen Zugriffsstufen
 - Gegner mit unterschiedlichen Schild- und Lebenswerten

Methoden

- Methoden als konkrete Aktionen von Objekten dargestellt
- Bestimmte Probleme können nur durch Ausführung der passenden Aktion gelöst werden
- Beispiele:
 - Öffnen eines Sicherheitssystems
 - Aktivieren eines Generators
 - Reparieren eines beschädigten Moduls
 - Neustarten eines Computersystems

Vererbung

- Verschiedene Robotertypen und Sicherheitssysteme basieren auf gemeinsamen Grundfunktionen
- Spezialisierte Varianten erweitern Funktionen um zusätzliche Fähigkeiten
- Beispiele:
 - RepairBot
 - SecurityBot
 - CargoBot

Polymorphismus

- gleiche Interaktion kann bei unterschiedlichen Objekten verschiedene Reaktionen hervorrufen
- Spieler*innen lernen, dass identische Befehle je nach Objekttyp unterschiedliche Ergebnisse erzeugen
- Beispiele:
 - Interaktion mit einer Tür öffnet diese
 - Interaktion mit einem Terminal zeigt Informationen an
 - Interaktion mit einem Generator aktiviert die Energieversorgung
 - Interaktion mit einem Aufzug startet den Transport zwischen Bereichen.

Kapselung

- Kritische Systeme können nicht direkt manipuliert werden
- Stattdessen müssen die Spieler*innen vorgesehene Schnittstellen verwenden
- Beispiele:
 - Verschlussene Türen können nur über entsprechende Sicherheitssysteme entsperrt werden
 - Energieanlagen müssen über Kontrollterminals aktiviert werden
 - Geschützte Datenmodule benötigen spezielle Zugriffsschlüssel

8. SPIELERFAHRUNG

Die Spielerfahrung steht im Zentrum des Projektes, da sie maßgeblich beeinflusst, wie motivierend, verständlich und zugänglich die Vermittlung objektorientierter Programmierkonzepte wahrgenommen wird. Die Gestaltung der Spielerfahrung verbindet Lernmechaniken, Exploration und interaktive Problemlösung innerhalb einer atmosphärischen Science-Fiction-Umgebung.

8.1 Immersionselemente

Damit Spielende möglichst intuitiv in die Spielwelt eintauchen können, werden verschiedene Elemente zur Unterstützung der Immersion eingesetzt.

Zentrale Immersionselemente:

- **Atmosphärische Raumstation-Umgebung:**
 - Futuristische 2D-Pixelotik mit klarer und verständlicher visueller Gestaltung
 - Unterschiedliche Bereiche mit jeweils eigenem visuellem Charakter
- **Akustische Immersion:**
 - Elektronische Ambient-Musik und Systemgeräusche
 - Audiofeedback für Interaktionen und Spielereignisse
 - Dynamische Audioübergänge zwischen verschiedenen Spielbereichen
- **Interaktive Spielwelt:**
 - Terminals, Türen und Systeme reagieren dynamisch auf Spielerinteraktionen
 - Interaktive Objekte erwecken ein lebendiges Raumstation-Feeling
- **Visuelles Feedback:**
 - Hervorhebung interaktiver Systeme und Objekte
 - Animationen und Effekte zur Unterstützung der Spielerführung
 - Informationsboxen als Hinweise auf Ereignisse oder Interaktionen

8.2 Bewegungssystem

Die Fortbewegung ist ein zentraler Bestandteil der Exploration der Raumstation und soll intuitiv sowie leicht verständlich gestaltet sein.

- Freie 2D-Bewegung innerhalb der Spielwelt (links, rechts, hoch, runter)
- Steuerung über Tastatur oder Touch-Eingaben
- Einfache Animationen für Charakterbewegung
- Fokus auf Zugänglichkeit und präzise Steuerung

8.3 UI/UX Design

Die Benutzeroberfläche soll übersichtlich, minimalistisch und funktional gestaltet werden, damit die Lern- und Gameplay-Elemente verständlich vermittelt werden können.

Zentrale Gestaltungsmerkmale:

- Minimalistische und gut lesbare UI
- Kontextabhängige Informationsfenster
- Klare Symbole und visuelle Hinweise
- Schrittweise Einführung neuer Mechaniken und Systeme
- Fokussierung auf intuitive Bedienung

9. SPIELVERLAUF

Der Spielverlauf von Deep Space Debugger basiert auf einer gameplayorientierten Lernstruktur, bei der Spielende die Raumstation schrittweise erkunden, technische Probleme analysieren und mithilfe objektorientierter Mechaniken lösen. Der Fortschritt erfolgt durch Exploration, Problemlösung und das Freischalten neuer Bereiche.

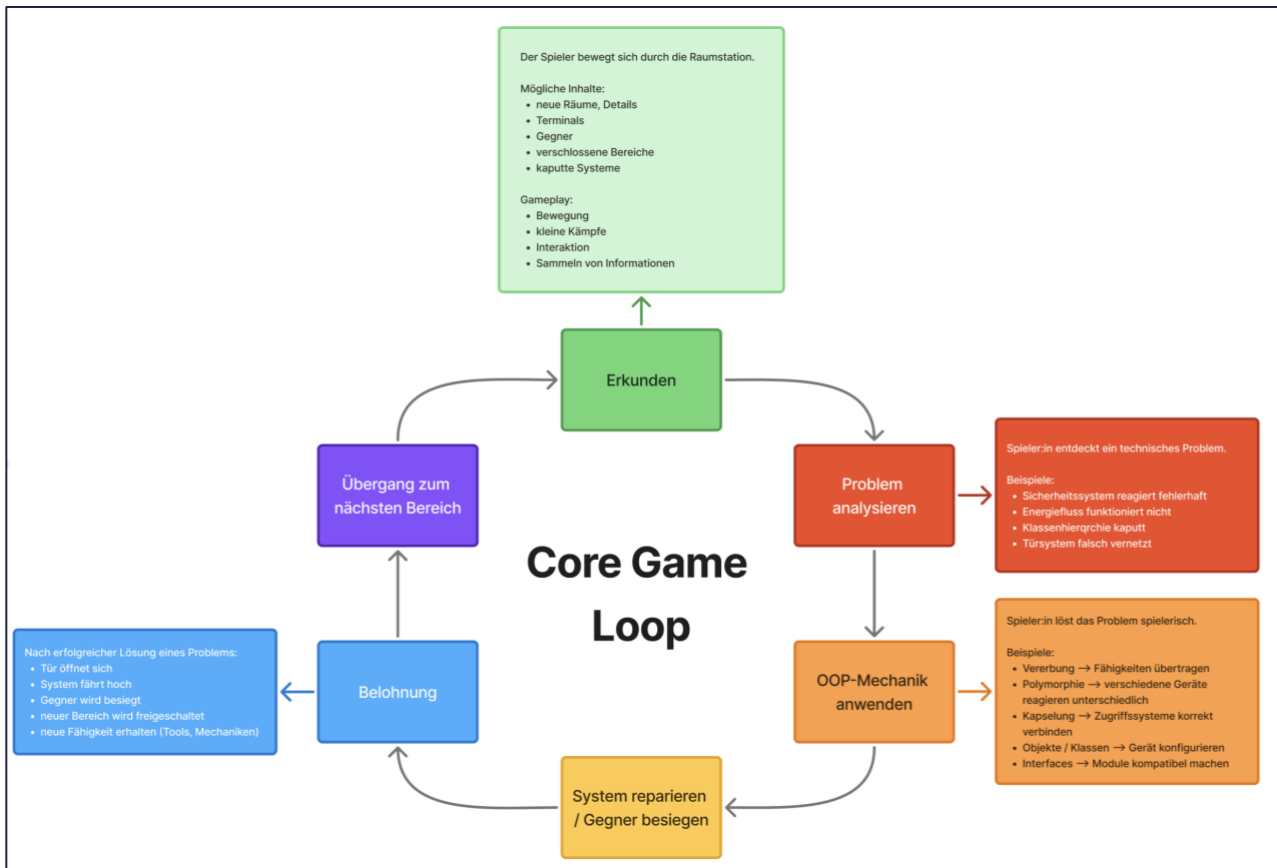
9.1 Grundstruktur: Bereichsbasierter Fortschritt

Das Spiel besteht aus **mehreren Bereichen der Raumstation**, die nacheinander freigeschaltet werden. Jeder Bereich vermittelt unterschiedliche Konzepte des objektorientierten Paradigmas und erweitert die Spielmöglichkeiten.

Phase	Ort	Fokus & Lernziel	Kernelemente	Dauer
Intro	Aufwachraum	Einführung in Steuerung, Atmosphäre & erste Interaktionen	Notfallmeldung, erste Terminals, Bewegung	3-5min
I	Energieversorgung	Einf. in Klassen und Objekte	Interaktive Systeme, erste Reparaturen	5-10min
II	Sicherheitssysteme	Verständnis von Vererbung & Systemtypen	Türen, Zugriffssysteme	5-10min
III	Robotikbereich	Polymorphie & unterschiedliche Objektinteraktionen	Gegner, variable Geräteinteraktionen	5-10min
IV	Kommunikationszentrum	Objektbeziehungen & Systemverknüpfungen	Netzwerk- & Systemrätsel	5-10min
V (Finale)	Biosphäre	Kombination aller OOP-Mechaniken	Lebenserhaltung, komplexe Systeminteraktionen	5-10min

9.2 Core-Game-Loop

Der Core-Game-Loop beschreibt den grundlegenden Ablauf der Spieleraktivitäten innerhalb des Spiels und strukturiert den kontinuierlichen Spielfortschritt.



9.3 Fortbewegung und Übergänge

Die Raumstation wird schrittweise erkundet und ist bewusst strukturiert aufgebaut, um Orientierung und Lernfortschritt zu unterstützen.

Spielweltstrukturierung:

- Freie 2D-Bewegung innerhalb der jeweiligen Bereiche
- Kompakte und übersichtliche Levelstrukturen
- Freischaltung weiterer Bereiche durch Fortschritt und Problemlösung

Übergänge:

- Verschlossene Türe oder andere Sicherheitssysteme begrenzen den Fortschritt
- Rätsellösung / Reparaturen notwendig für das voranschreiten in weitere Bereiche
- Neuer Bereich = Belohnung

9.4 Spielabschluss

Nach erfolgreicher Reparatur der zentralen Systeme und somit der Abschluss der letzten Mission wird die Raumstation abschließend vollständig reaktiviert. Damit endet die prototypische Spielerfahrung.

Der Abschluss vermittelt den Spielenden:

- Den erfolgreichen Einsatz der erlernten OOP-Konzepte
- Spielerischen Fortschritt innerhalb der Raumstation
- Verbindung zwischen theoretischer Problemlösung und praktischer Anwendung

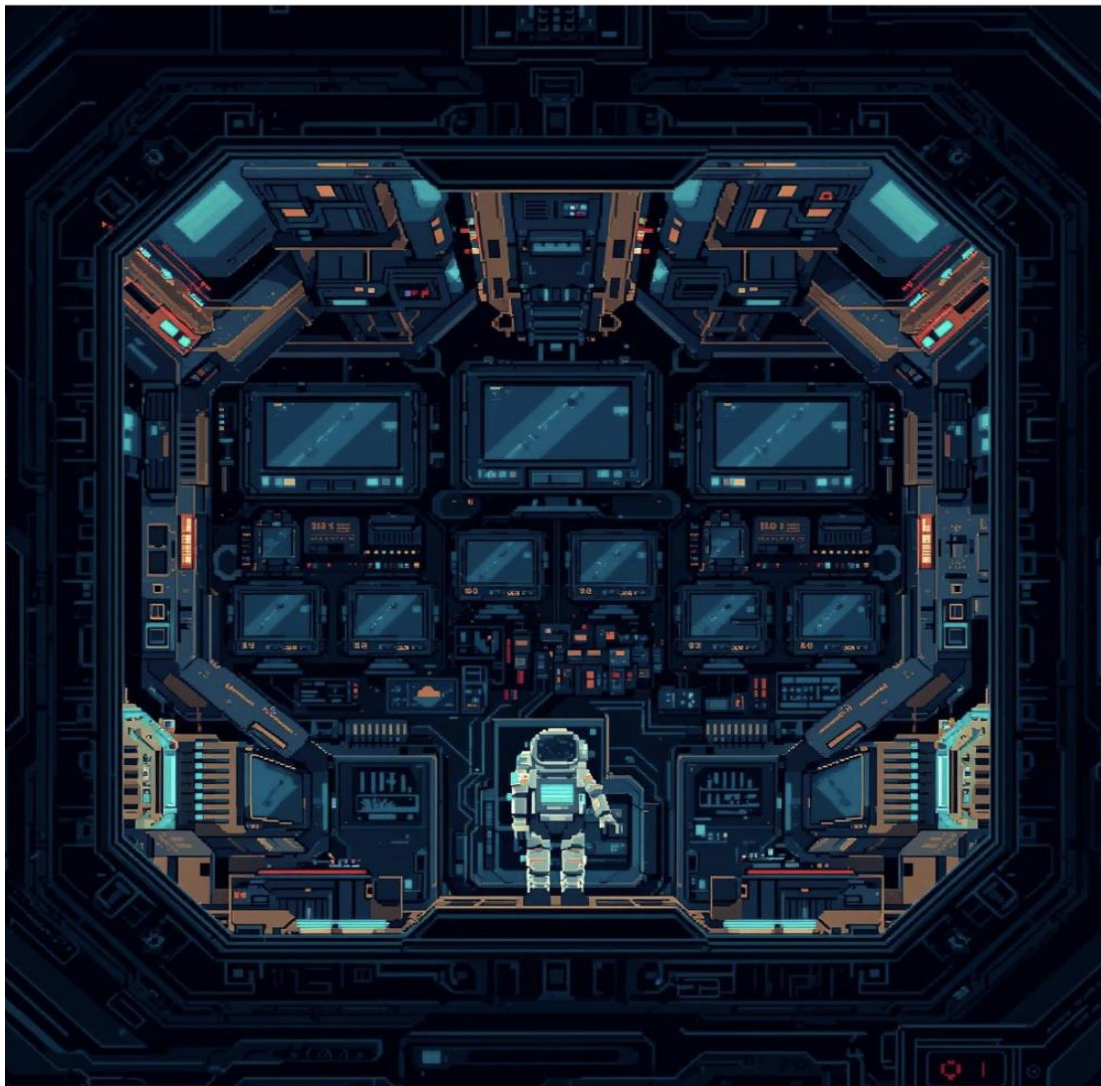
10. STORY & SETTING

Das Spiel Deep Space Debugger spielt auf einer beschädigten Raumstation im tiefen Weltraum. Nach einem schwerwiegenden Systemausfall sind zentrale Bereiche der Station außer Betrieb. Die Spieler:innen übernehmen die Rolle eines Systemtechnikers, welcher als Notfallmaßnahme vom System aus dem Kälteschlaf geweckt wird, um der Ursache für die Störungen zu untersuchen und die verschiedenen Stationssysteme schrittweise wiederzustellen.

10.1 Wichtige Orte

- **Aufwachraum / Kälteschlafkapsel: (Tutorial + Einführung)**
 - Einstieg in das Spiel und Einführung in die Grundlagen
 - Vermittlung der Ausgangssituation und erster Mechaniken
- **Energieversorgung:**
 - Einführung in Klassen und Konzepte als OOP-Konzepte
 - Reparatur grundlegender Systeme
 - Tür zum Sicherheitsbereich öffnet sich
- **Sicherheitssysteme:**
 - Einführung in Vererbung und Spezialisierungen
 - Freischaltung neuer Stationsbereiche
- **Robotikbereich:**
 - Anwendung von Polymorphie und verschiedener Objektverhalten
 - Interaktion mit weiterem System und Gegnern
- **Kommunikationszentrum:**
 - Fokus auf Objektbeziehungen und Systemverknüpfungen
 - Analyse komplexer technischer Zusammenhänge
 - Meldung des Notfalls an zentrale Einheit
- **Biosphäre (Finale):**
 - Kombination aller zuvor erlernten OOP-Konzepte
 - Wiederherstellung aller wichtigen Systeme und Stabilisierung der Station

11. MOODBOARDS UND IMPRESSIONEN





12 QUELLEN

12.1 Bildquellen

- Nutzung von KI generierten Bildern
- Canva AI Bildgenerierung
- <https://i.pinimg.com/564x/a4/78/50/a47850db05b800f48bc14a1c32f14aa2.jpg>
- <https://preview.redd.it/spacestation-progress-working-on-a-big-spaceship-tileset-v0-tdlbobqgftqb1.png?width=640&crop=smart&auto=webp&s=c731ebe85547aec0a0d4d62db7437974896e0fee>
- <https://img.itch.zone/aW1nLzk3ODI2MjAuanBn/315x250%23c/iJBubK.jpg>